



UNIVERSITY OF
WATERLOO

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

ECE-124 LAB MANUAL

V3.0

LAB 3

Course: ECE-124 Digital Circuits and
Systems(with Verilog)

LAB3: NEW Verilog Style for Combinational or Sequential Logic Circuits

In this lab, a new project for combinational and sequential logic will be designed in Verilog. The project for this lab session is a simple Energy Monitoring Controller that could be used in a home.

There will be two ways to test the design. The first will be with a Simulation environment and the second will be with an in-design “Tester” approach.

For the Tester, a new style of Verilog coding (BEHAVIORAL) will be introduced. A sequential logic block (HVAC) will also be created using the Verilog Behavioural style.

The final design must be demonstrated during the next Lab session.

1.1 Lab3 Intended Learning Outcomes

By the end of this lab, students should be able to:

- 1) UNDERSTAND the two types of Verilog Statement Domains (Concurrent and Sequential)
- 2) UNDERSTAND the Concurrent Statement Domain for Combinational logic design
- 3) UNDERSTAND the Sequential Statement Domain for Combinational / Sequential logic design
- 4) UNDERSTAND how a technique for clock speed control with parameter passing
- 5) UNDERSTAND how extra ports can be added for Functional Simulations of internal signals

1.2 Lab3 Outline

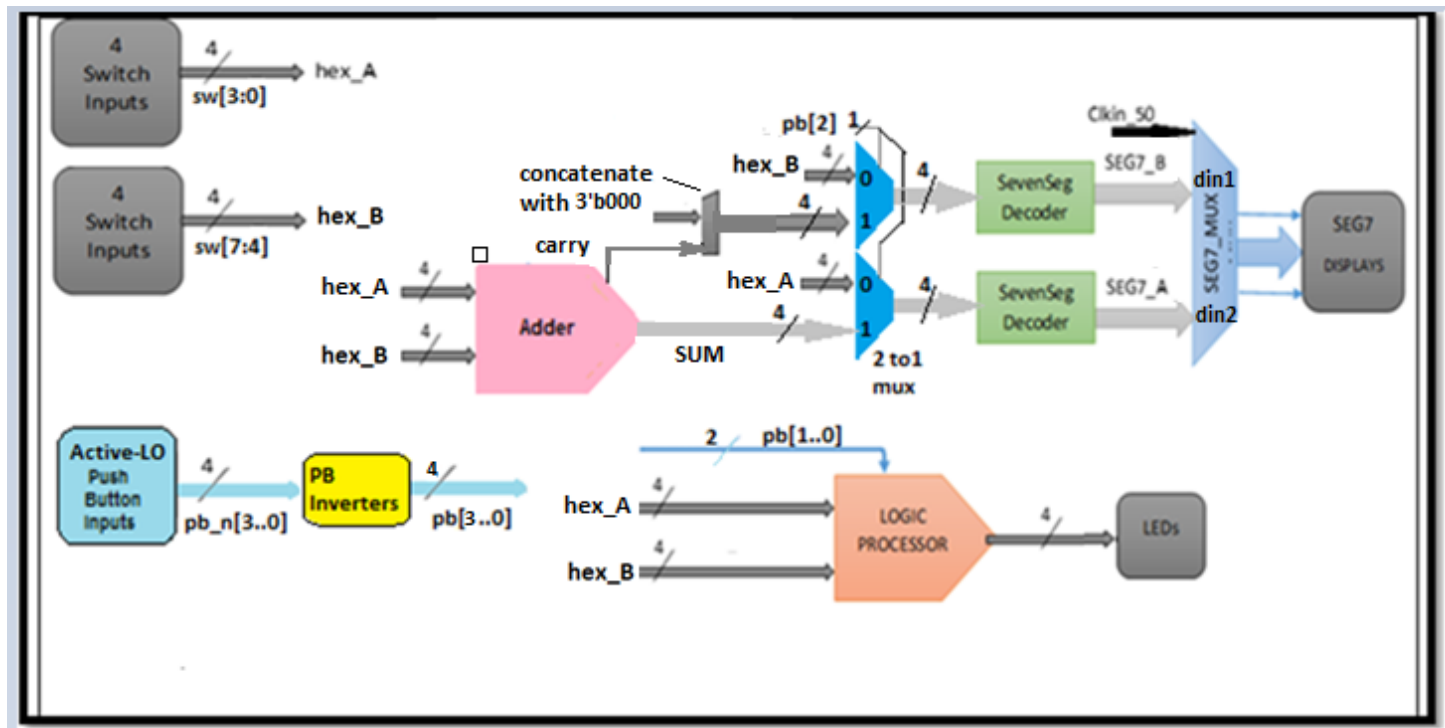
The following topics will be presented:

1. Short Review of Lab2 (Dataflow and Structural coding styles of Verilog)
2. Project Setup for Lab3
3. Lab3 Starter Design
4. Lab3 Topic: New Verilog Component - Magnitude Comparator
5. **Lab3 Part A:** Creating/Testing a 4 Bit Magnitude Comparator (Dataflow/Structural Verilog)
6. Lab3 Topic: Verilog STATEMENT DOMAINS – Concurrent vs Sequential
7. Lab3 Topic: Using Sequential Domain for Combinational logic or Sequential Logic
8. **Lab3 Part B:** Creating Combinational Tester Design and Sequential Up/DOWN counter
9. Lab3 Topic: Compiler Directives and Parameter Passing use in a Design
10. **Lab3 Part C:** Employing Compiler Directives and adding a 2_to_1 MUX, 4-bit Inverter Modules
11. **Lab3 Part D:** Creating a Home Energy Monitor with an on-chip Tester (Lab3 Demo)

1.3 Lab3 Activities

1.3.1 Short Review of Lab2:

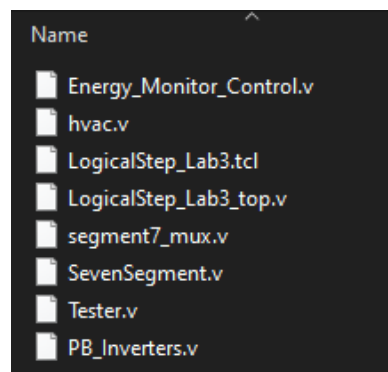
During Lab2, we developed some Verilog knowledge on using the Verilog “Dataflow” and “Structural” styles of coding. We learned how to reuse previously developed Verilog files as subordinate functions. (Multiplexers, Adder and Logic_Processor) were instantiated in the LogicalStep_Lab2_top Verilog design file. There was also a brief coverage of how to use CONCATENATION in Verilog.



The INSTANCES were then “wired” together using declared signals (wires).

1.3.2 Lab3 Topic: Project Setup

Start Lab3 (like way that was done in earlier labs) by creating a new project folder on the C: drive. Using the Windows File Explorer go to your ECE-124 folder directory. Go to LEARN and download the Lab3 Zipped folder “Lab3” into the ECE-124 folder. Do the “Extract here” mode for extracting the contents. This will create the Lab3 project folder and its contents.



You should end up with a file directory having ONE Lab3 level (NOT TWO) inside the ECE-124 parent folder.

Start up the Quartus Prime platform and begin a new project (Using **FILE>New Project Wizard**).

Click **NEXT** to go to the second dialog window page.

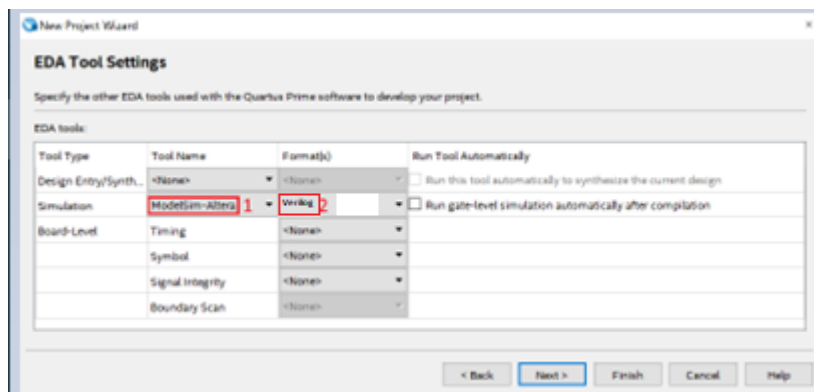
Enter the new Project parameters:

Project Folder: **C:/users/<name>/ECE-124/Lab3**

Project Name: **LogicalStep_Lab3**

Project Top Level: **LogicalStep_Lab3_top**

Then click **NEXT** on the New Project Wizard Dialog Window. Then click **NEXT** again (with Empty Project). Click **NEXT** on dialog windows for Family, Device & Board Settings until you reach the following window (**EDA Tool Settings**). Then choose **Modelsim-Altera** and **Verilog** from the drop-down menu according to the following figure:



Then Click FINISH.

THIS NEXT STEP IS IMPORTANT (.tcl script)

Next, since we are now creating a design for an actual FPGA board in the Lab, we must set up the target FPGA family (MAX10) device and link the pin names in the LogicalStep_Lab3_top Entity section to actual pin locations that are used on the FPGA board. In Quartus, go to the menu bar and select **TOOLS> Tcl Scripts** and then browse to the **LogicalStep_Lab3.tcl** file in the Project folder.

The TCL script is used to assign the FPGA device type (MAX10) and assign pin locations for the LogicalStep_Lab3 project.

1.3.2.1 Lab3 Starter Design

The top-level design file (LogicalStep_Lab3_top.v) is shown in the next couple of pages. Notice in the screenshot that the input and output pins are declared in the Module declaration statement for the LogicalStep_Lab3_top file.

```

1  module LogicalStep_Lab3_top (
2      input      clk_in_50,
3      input      [3:0] pb_n,
4      input      [7:0] sw,
5      output     [7:0] leds,
6
7      output     [6:0] seg7_data, // 7-bit outputs to a 7-segment
8      output     seg7_char1, // seg7 digit1 selector
9      output     seg7_char2 // seg7 digit2 selector
10 );
11

```

```

// INTERNAL SIGNALS USED IN THE DESIGN
// declarations associated with the global clock, push-buttons and switches
wire clk_in;
wire [3:0] pb, hex_A, hex_B;

// declarations associated with the seg7 signals
wire [6:0] seg7_A, seg7_B;

//-----
// Here the circuit begins

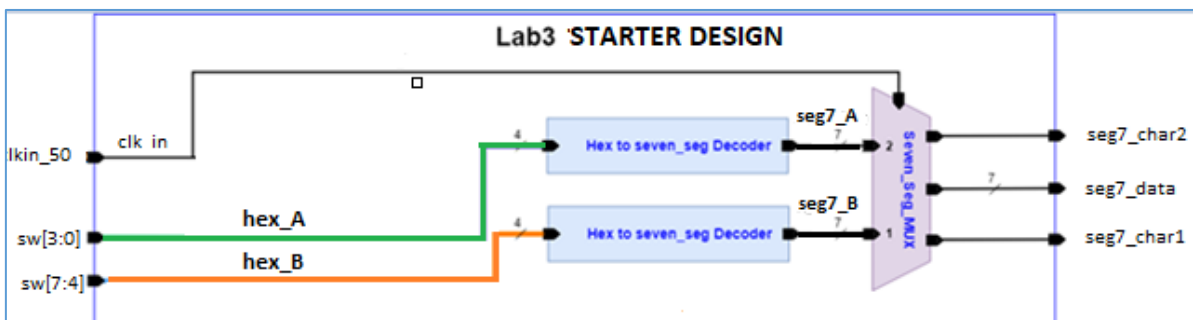
// hookup up the clock and switches
assign clk_in = clk_in_50;
assign hex_A = sw[3:0];
assign hex_B = sw[7:4];

// hookup to hex signals to the 7seg decoders (you may use your decoders from Lab2)
SevenSegment u0 (
    .hex (hex_A), .sevensseg (seg7_A)
);
SevenSegment u1 (
    .hex (hex_B), .sevensseg (seg7_B)
);

// hookup to seven_segment_mux function
segment7_mux u2 (
    .clk (clk_in), .din2 (seg7_A), .din1 (seg7_B), .dout (seg7_data), .dig2 (seg7_char2), .dig1 (seg7_char1)
);

//-----
// PART A build
//-----
// Part B build
//-----
// Part C build
//-----
// Part D build
//-----
);
endmodule

```



This Starter design for Lab3 can be compiled for a download to the LogicalStep board.

1.3.3 Lab3 Topic: New Verilog Component – Magnitude Comparator

A magnitude comparator accepts two input variables and determines how they compare to each other in terms of magnitude. There are three outputs: input A is greater than input B; input A is equal to input B; and input A is less than input B.

1.3.4 Lab3 PART A: Creating/Testing a 4-Bit Magnitude Comparator (DATAFLOW / STRUCTURAL Verilog)

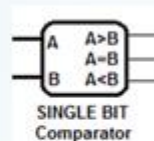
For Lab3 a 4-bit Magnitude Comparator is to be designed with **GATES** (i.e. no IF or CASE statements) using two levels of hierarchy. The lower level (DATAFLOW) will just compare the magnitude of two single bits and then each one passes along its three results (A>B, A=B, A<B) to the upper level. The upper level will use multiple instances of the lower level (component). This upper level will be largely STRUCTURAL in Verilog coding. However, alongside the Structural portion will be another set of DATAFLOW implementations to create the second level Verilog outputs. These DATAFLOW-based outputs at this level are the overall A>B, A=B, A<B results for the four-bit operands of A and B.

The logic that is to be built in Lab3 project requires a 4-bit Magnitude Comparator to determine whether more energy is required for heating or cooling a home.

So...you guessed it... we must make up one for the project. See below.

Start by creating a Single bit Magnitude Comparator Truth Table with single bit inputs for A and B. This is the lower level talked about earlier. Then create three output columns (one for each of A>B, A=B, A<B) and then use logical reasoning to determine which conditions force these outputs to activate.

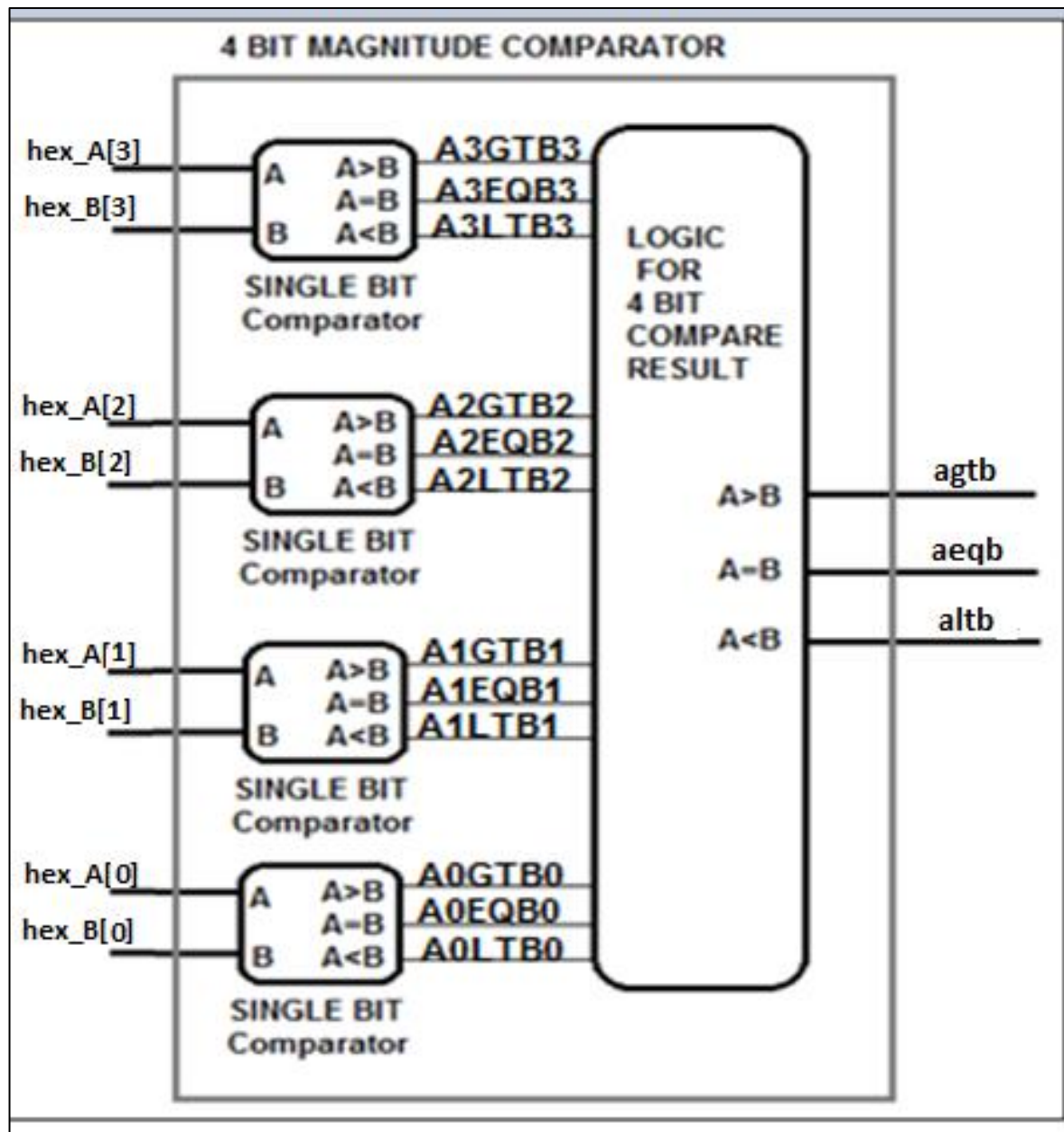
Input A	Input B	A>B	A=B	A<B
0	0			
0	1			
1	0			
1	1			



Fill in the values to a Truth Table to determine when each output is ON.
Create a file for a 1-bit Comparator (Comp1) that has two inputs and three outputs.
Write Boolean equations for the three outputs based on the values of the two inputs.

Create a Single-bit comparator Dataflow Verilog file (**Comp1.v**) with the above information in its design using **Boolean equations for the outputs**.

Later, this Verilog function can be instantiated as a 1-bit Comparator in a higher-level Verilog file.



Next, create a higher-level Verilog file (**Comp4.v**) which includes four instantiations of **Comp1**. Then declare signals to connect to each set of 3 outputs from each of the Comp1 instances. You may wish to create signals such as AGTB[3:0]), AEQB[3:0] etc. Then each bit of these signal groups can be specified by an index value within the vector (i.e.: AGTB[2]).

In the Comp4 Verilog file, make use of all the individual comparisons coming from the instances of Comp1. This logic is to be written in **BOOLEAN form** as well.

In this new Verilog file, the function outputs can be created by using the following table below.

Comparison Inputs from 1-Bit Comparators												4-Bit Comparator Outputs		
A3<B3	A3=B3	A3>B3	A2<B2	A2=B2	A2>B2	A1<B1	A1=B1	A1>B1	A0<B0	A0=B0	A0>B0	A<B	A=B	A>B
0	0	1	X	X	X	X	X	X	X	X	X	0	0	1
1	0	0	X	X	X	X	X	X	X	X	X	1	0	0
0	1	0	0	0	1	X	X	X	X	X	X	0	0	1
0	1	0				X	X	X	X	X	X			
0	1	0							X	X	X			
0	1	0							X	X	X			
0	1	0												
0	1	0												
0	1	0	0	1	0	0	1	0	0	1	0	0	1	0

Start with the highest-order bit in the group of bits being compared. For example:

If $A3 > B3$ is TRUE then the lower bits of A2, B2, A1, B1, A0, B0 do not need to be considered further for the 4-bit $A > B$ equation. Use the table below (X means “Don’t Care”).

Same thing if $B3 > A3$ (which is of course the same thing as $A3 < B3$).

However, if $A3 = B3$ is TRUE then you must look to the comparison of the next lower bits ($A2 > B2$) and so on. Remember the $A > B$, $A = B$, $A < B$ outputs are all mutually exclusive (ONLY ONE CAN BE ON for any given row in the table).

Your design for the 4-bit Comparator (Comp4) will be based on that logical approach. Fill in the blanks for the inputs and outputs for the 4-bit, Comparator function table above. Then write **Boolean equations for the three Comparator outputs inside your Comp4 file based on the inputs**.

After the design of the Comp4 is completed; open the LogicalStep_Lab3_top.v file and modify it in the following manner by adding the following into the PART A build section:

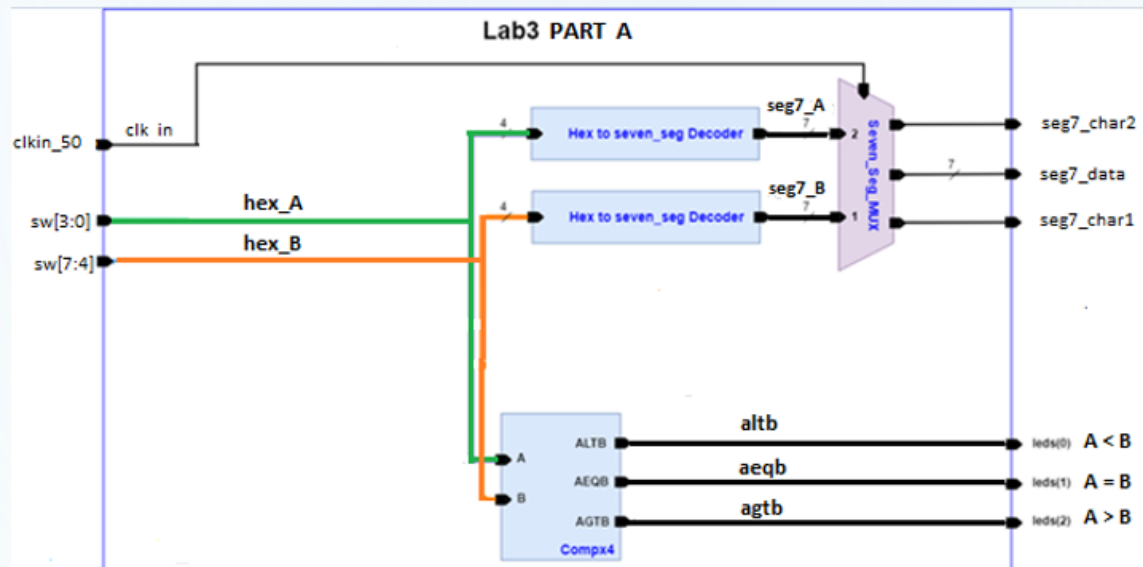
```
//-----
// PART A build

// declarations associated with the Comparator signals
wire      aeqb, agtb, altb;

//add your 4bit magnitude comparator here
comp4x    U3 (
    .hex_A (hex_A), .hex_B (hex_B), .A_eq_B (aeqb), .A_gt_B (agtb), .A_lt_B (altb)
);

// hookup up 3 leds for the comparator testing
assign leds[0] = altb;
assign leds[1] = aeqb;
assign leds[2] = agtb;
//-----
```

LAB 3 TOPIC: PART A BUILD



Compile and download the LogicalStep_Lab3_top design into the LogicalStep board.

Three of the LEDs will be used to validate the operations of your 4-Bit Magnitude Comparator design.

A figure below shows a possible approach for stimulus etc.

Lab 3 PART A: TESTING the Comparator

TEST 1	
Sw [7..4]	X 0 1 2 3 4 5 6 7 8 9 A B
Sw [3..0]	X 1
TEST 2	
Sw [7..4]	X 0 1 2 3 4 5 6 7 8 9 A B
Sw [3..0]	X 2
TEST 3	
Sw [7..4]	X 0 1 2 3 4 5 6 7 8 9 A B
Sw [3..0]	X 5
TEST 4	
Sw [7..4]	X 0 1 2 3 4 5 6 7 8 9 A B
Sw [3..0]	X 8

Check for
led0 as A<B,
led1 as A=B,
led2 as A>B.

THEN
Repeat but
SWAP the
input test
patterns

For your design, you can compare the hex_A and hex_B values and see if the correct led activates from the 4-bit Magnitude Comparator. Only one led should be active at a time (AGTB or AEQB or ALTB). If an incorrect result is displayed, you can return to the designs that you created for Comp1 and Comp4 to make corrections.

1.3.5 Lab3 Topic: Verilog STATEMENT DOMAINS – Concurrent vs Sequential

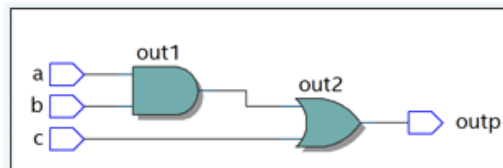
There are two types of Verilog statement domains. These are the Concurrent Statement domain and the Sequential Statement domain. These domains are processed differently during the time when the process is being compiled or “synthesized” into logic.

The Concurrent Statement domain Verilog is like the Dataflow / Structural styles used in Labs 1 and 2. The statements are synthesized directly into logic during compilation and all logic runs concurrently. The order of the statements does not matter.

```
module gate (
  input a,b,c,
  output outp
);
  wire out1, out2;

  assign out1 = a & b;
  assign out2 = out1 | c;

  assign outp = out2;
endmodule
```

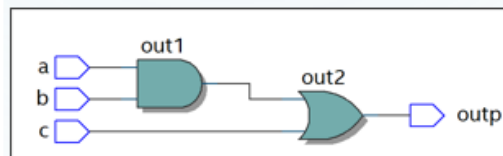


Concurrent Statements are for **COMBINATIONAL LOGIC ONLY** and they use the Assign statements. In the Concurrent statement domain, the statements can be in any order.

```
module gate (
  input a,b,c,
  output outp
);
  wire out1, out2;

  assign out2 = out1 | c;
  assign out1 = a & b;

  assign outp = out2;
endmodule
```



The Sequential Statement domain Verilog is handled differently during the compile time. The logic descriptions can take more of an “algorithmic” kind of format. This form of Verilog is called Behavioural Verilog.

At compile time, the sequential statements are evaluated in the order sequence and the compiler creates the synthesized logic for all possible process outcomes.

In this domain, there are TWO types of Procedural Blocks. These are the INITIAL and the ALWAYS Procedural Blocks.

Initial Procedural Blocks are made to run **ONLY ONCE** and are used in Test Bench designs.

INITIAL Procedural Blocks are **NON-SYNTHESIZABLE** since they usually use a time-based parameter details for simulation testing. We will not be using this form in the lab exercises for this course. This course uses the graphical-based stimulus waveforms defined by the user in the Quartus simulator.

For this course, the sequential statements of this domain are confined to be inside an ALWAYS Procedural block.

ALWAYS Procedural Blocks, when compiled, will run repeatedly with the synthesized logic created.

always @ (sensitivity list)

begin

sequential statements

...

sequential statements

end

always@: sets up the always procedural block

sensitivity list: signals that force block to be updated whenever those signals change.

-for combinational logic (e.g.: gate functions) all input signals for sequential statements must be present in this list.

-for sequential logic, (e.g.: FF or Latch functions) the process primarily uses synchronizing signals (like clocks etc.). These will be covered later.

begin_keyword for the start of the block

Sequential statement(s):

- Output signals are variables driven by the Always Procedural Block are declared OUTSIDE of the Always Procedural block. They MUST have the **reg** datatype declaration outside of the Procedural Block where those signals connect to other logic.
- The statements are executed in sequence. Sequential statements can include a new set of constructs for logic design. Examples can include statements such as “IF-ELSE”; “CASE” statements etc.
- For Combinational Logic, “blocking” assignments (=) are used
- For Sequential Logic, “non-blocking” assignments (<=) are used.

end keyword for the end of the block.

1.3.6 Lab3 Topic: Using Sequential Domain for Combinational logic or Sequential Logic

The Sequential Statement domain can be used for creating either kind of digital logic (Combinational or Sequential). Please don't conflate Sequential Statements and Sequential Logic. They are two different things.

Below are some examples of how Sequential statements inside an ALWAYS Procedural block can create Combinational logic. A variable "i" is declared outside of the blocks with a **reg** datatype.

All inputs to the ALWAYS Block are in the Sensitivity List; Sometimes just an asterisk (*) is used.

BLOCKING assignments (=) in a Procedural Block are used to execute the statements sequentially.

The order of statements matters. This is necessary for COMBINATIONAL LOGIC.

```

module gate (
  input a,b,c,
  output outp
);
  reg i;
  always@(a,b,c)
  begin
    i = a & b;
    i = i | c;
    i = i ^ c;
    i = i & c;
  end
  assign outp = i;
endmodule

```

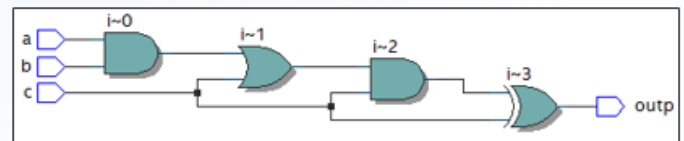
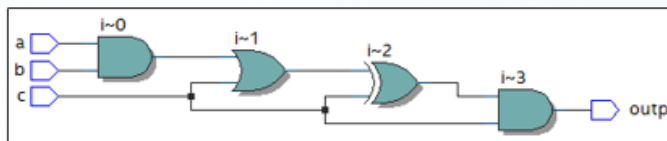
Sequential Statements for COMBINATIONAL LOGIC must use BLOCKING (=) assignments.

NOTE how the ASSIGN statement is outside of the Always block. It is in the Concurrent statement domain.

```

module gate (
  input a,b,c,
  output outp
);
  reg i;
  always@(*)
  begin
    i = a & b;
    i = i | c;
    i = i & c;
    i = i ^ c;
  end
  assign outp = i;
endmodule

```



The figure below shows how Combinational logic MUX can be created with an If-Else Sequential statement.

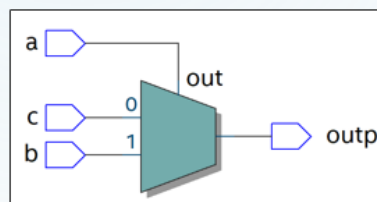
Below, "out" is an externally declared variable signal with the datatype of reg

```

module gate (
  input a,b,c,
  output outp
);
  reg out;
  always@(a,b,c)
  begin
    if (a == 1'b1)
      out = b;
    else
      out = c;
  end
  assign outp = out;
endmodule

```

Conditional statements such as "IF – ELSE" or "CASE" statements can also be used to create Combinational logic but BE CAREFUL to make sure that outputs driven by the Procedural block are defined for all conditional clauses to avoid latch inference. assignments



Sequential Statements for COMBINATIONAL LOGIC must use BLOCKING (=) assignments.

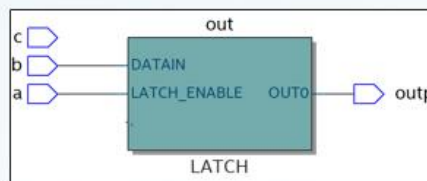
BUT YOU MUST BE CAREFUL WHEN USING IF statements. Below is an example where an assignment to a variable is made when a particular input is ACTIVE. But there is no ELSE clause included to cover the case when that input is NOT ACTIVE. An **inferred latch** is created by the compiler in this case.

```

module gate (
  input  a, b, c,
  output outp
);
  reg out;
  always@(a,b,c)
begin
  if (a== 1'b1)
    out = b;
  end
  assign outp = out;
endmodule

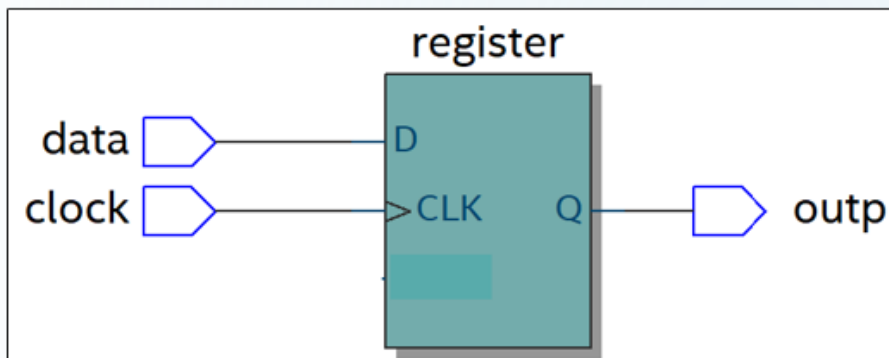
```

IF statements without an ELSE clause CREATE INFERRED LATCHES
Or by signals not being assigned on all clauses of an IF- ELSE statement



For creating Sequential Logic, the Sequential Statement domain MUST be employed. Sequential logic involves storage elements like flip-flops etc. These are used in counter and registers and so on.

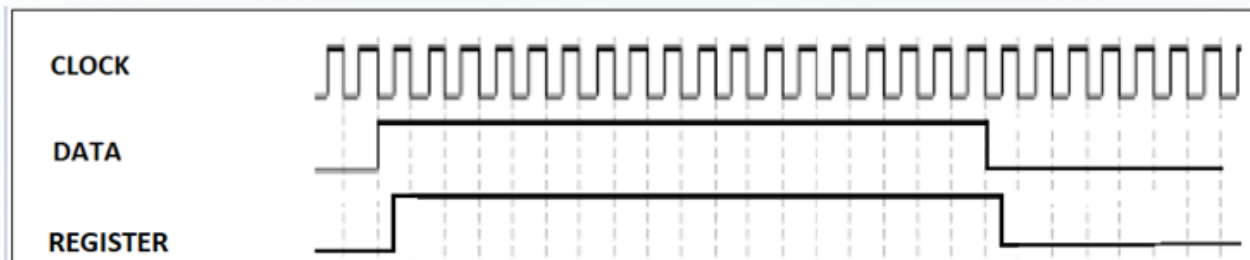
For the greatest timing precision in the operation of these storage elements, a Verilog “posedge” operator is used. The sensitivity list in the ALWAYS statement includes a clock with this posedge operator. See below:



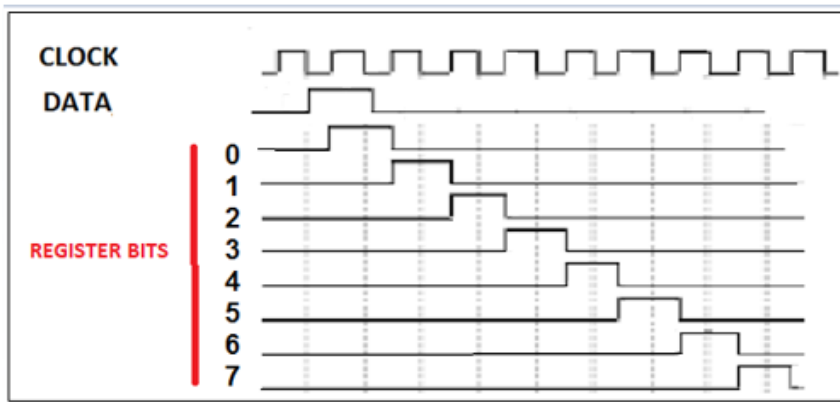
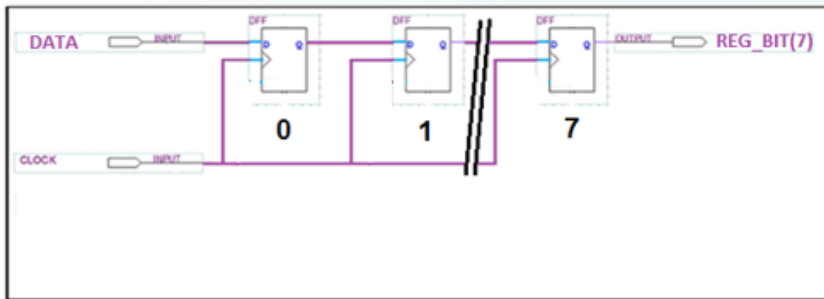
```

module ff_top (
  input data, clock,
  output outp
);
  reg register;
  always@(posedge (clock))
begin
  register <= data;
  end
  assign outp = register;
endmodule

```



If a number of these flip-flops are connected in series the following operation can be seen:

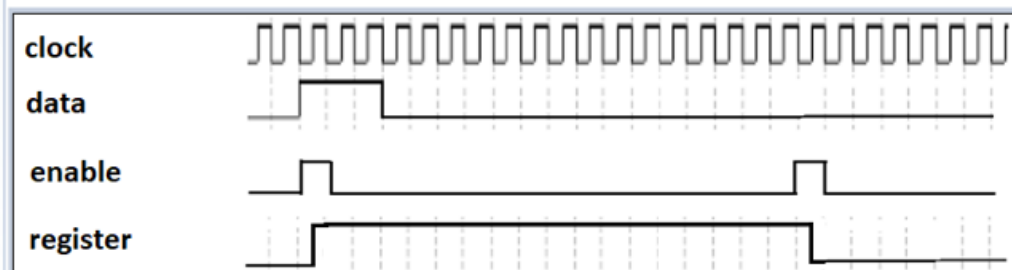
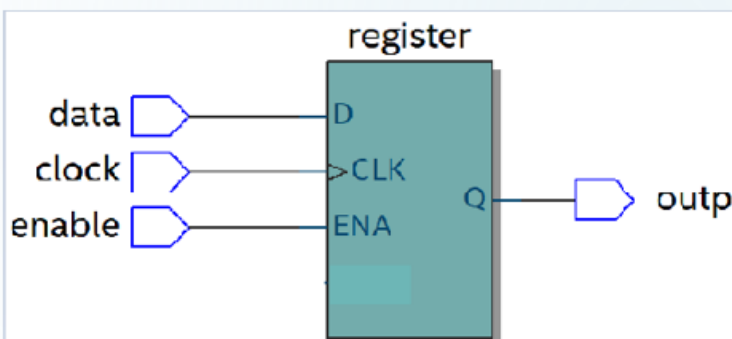


```

module ff_top (
    input data,
    input clock,
    output [7:0] outp
);
    reg [7:0] register;
    always@(posedge (clock))
    begin
        register[7:0] <= {register[6:0], data};
    end
    assign outp = register;
endmodule

```

Another flavour of a storage element that one can use is a Flip-Flop with a Clock Enable. This extra function can be used to selectively capture data into the flip-flop.



```

module ff_top (
    input data, enable,
    input clock,
    output outp
);
    reg register;
    always@(posedge (clock))
    begin
        if (enable == 1'b1)
            register <= data;
        end
    assign outp = register;
endmodule

```

NOTE: Latch (or flip-flop) inference is intentionally used here.

```

module ff_top (
    input data, enable,
    input clock,
    output [7:0] outp
);
    reg [7:0] register;
    always@(posedge (clock))
    begin
        if (enable == 1'b1)
            register[7:0] <= {register[6:0], data};
    end
    assign outp = register;
endmodule

```

CLOCK
ENABLE
DATA_INPUT

REGISTER BITS

University of Waterloo

1.3.7 Lab3 Part B: Creating Combinational Tester Design and Sequential Up/DOWN counter

In Lab2 you saw that Functional Simulations are very useful during the testing of digital designs. However, when the design is moved to a production setting, a FASTER and more feasible means of testing each unit is sometimes required. The high production volume requires short and thorough testing times.

To achieve that, an “on-chip Tester” may be employed in the digital design. This function is normally “dormant” in the user application, but it can get used in the production or field servicing situations for design confirmation testing.

Testers can vary in complexity, performance, and scope. Quite often, they are designed with a high level of abstraction because it may not be critical as to what logic is used to implement the Tester design.

In the Tester design for Lab3, an ALWAYS Procedural block will be used as a basis to test the 4-bit Magnitude Comparator outputs against the Input1, Input2 values coming to the Tester. This design will be Combinational logic.

Fill in the missing **else If** statement conditions.

NOTE: Sequential Statements for COMBINATIONAL LOGIC always use BLOCKING (=) assignments.
Signals driven by the Procedural block must be present in all IF statement clauses

```

module Tester (
    input    mc_testmode,
    input    ileqi2, ilgti2, illti2,
    input [3:0] input1, input2,
    output reg test_pass
);
    reg eq_pass, gt_pass, lt_pass;
    always@(*)
    begin
        if ((input1 == input2) && (ileqi2 == 1'b1)) begin
            eq_pass = 1'b1;
            gt_pass = 1'b0;
            lt_pass = 1'b0;
        end
        else if(
            ) begin
            gt_pass = 1'b1;
            eq_pass = 1'b0;
            lt_pass = 1'b0;
        end
        else if(
            ) begin
            lt_pass = 1'b1;
            eq_pass = 1'b0;
            gt_pass = 1'b0;
        end
        else begin
            eq_pass = 1'b0;
            gt_pass = 1'b0;
            lt_pass = 1'b0;
        end
        test_pass = mc_testmode & (eq_pass | gt_pass | lt_pass);
    end
end

```

PLEASE NOTE - ABOUT IF STATEMENTS: For ALL signals being controlled by the IF statement clauses, make sure that the signal assignments are present in ALL IF STATEMENT CLAUSES. See the TESTER.v file above as an example (GTPASS, LTPASS, EQPASS). THIS PREVENTS INFERRED LATCHES.

Now a Sequential logic design (up/down counter) will be created by an ALWAYS Procedural block.

HVAC is a term that is known in the marketplace. It is used for **H**eating, **V**entilation and **A**ir **C**onditioning.

A design file for this function is included for your Lab3 PART B build. You are to complete the design of this file. It will emulate the operation of a Furnace/Air Conditioner temperature unit. The inputs to it are **clk**, **run**, **increase**, **decrease**. The hvac_sim input will be discussed later in the PART C build.

There is only one output port from the HVAC unit (which is **temp** [3:0]).

The HVAC unit will turn on only when the “run” is active. The “temp” output from the HVAC will change according to which of two qualifier signals is active (increase or decrease) when with the run signal being active.

The initial default starting value for the HVAC is at mid-scale (Hex 7). The temperature output value will continue to happen until the input **run** control signal goes inactive OR until one of two temp limits is reached (4'b0000 or 4'b1111). The HVAC MUST NOT go beyond those limits even if the run input signal is still active.

```
1 // clk input is the LogicalStep 50MHz clock input
2 module hvac #(parameter hvac_sim)
3
4   input          clk, run, increase, decrease,
5   output [3:0]    temp
6
7
8
```

A PARAMETER (hvac_sim) IS PASSED FROM A HIGHER LEVEL IN THE DESIGN

The variable “cnt” must increment when inputs for “run”, “increase” are both ON.

The variable “cnt” must decrement when inputs for “run”, “decrease” are both ON.

“cnt” must NOT overflow or underflow when reaching the range limits

```
// wire clk_2hz; // this clock is used for when the hvac counter is running on the
// LogicalStep board. The slow 2Hz clk makes the changes easy to observe.
reg hvac_clock; // this clock is used for when the hvac counter is running .
// in the Simulator LogicalStep board.
// The fast 50MHz clk makes the short time for simulation events easier to observe.
reg unsigned [3:0] cnt = 4'b0111; // variable (reg) hex value used in an always process
// initially set to mid-range.

reg [23:0] digital_counter; // output signal used in an always process and it must be a variable (reg)
// clk_divider process generates a 2Hz clock from the 50 Mhz clk
always@(posedge clk)
begin
  reg unsigned [23:0] counter;
  // Synchronously update counter
  counter = counter + 1;
  digital_counter = counter;
end
assign clk_2hz = digital_counter[23]; // slow 2Hz clock

always@(*)
begin
  if (hvac_sim == 1'b1) // hvac_sim is a multiplexer select control.
    hvac_clock = clk; // fast 50MHz clock
  else
    hvac_clock = clk_2hz; // slow 2Hz clock
  end
always@(posedge hvac_clock)
begin
  // update the cnt value based on inputs run, increase, decrease and the cnt value NOT
  // going above decimal 15 or below decimal 0..
  if
  //
  //
  else if
  //
  end
  assign temp = cnt;
endmodule
```

Add the following content to the LogicalStep_Lab3_top Part B Build section. This include new wire declarations and module instances. Some earlier build parts will need to be “commented out”.

1. Complete the Verilog coding for the Tester.v and hvac.v files
2. Add the following to the LogicalStep_Lab3_top.v file after the PART A BUILD section

```
//-----
// Part B build
wire [3:0] current_temp;

// COMMENT OUT THE SEVENSEGMENT U1 MODULE ABOVE (in the starter part of the file) AND USE THIS ONE FOR PART B <-----
SevenSegment U1 (
    .hex (current_temp), .sevenseg (seg7_B)
);

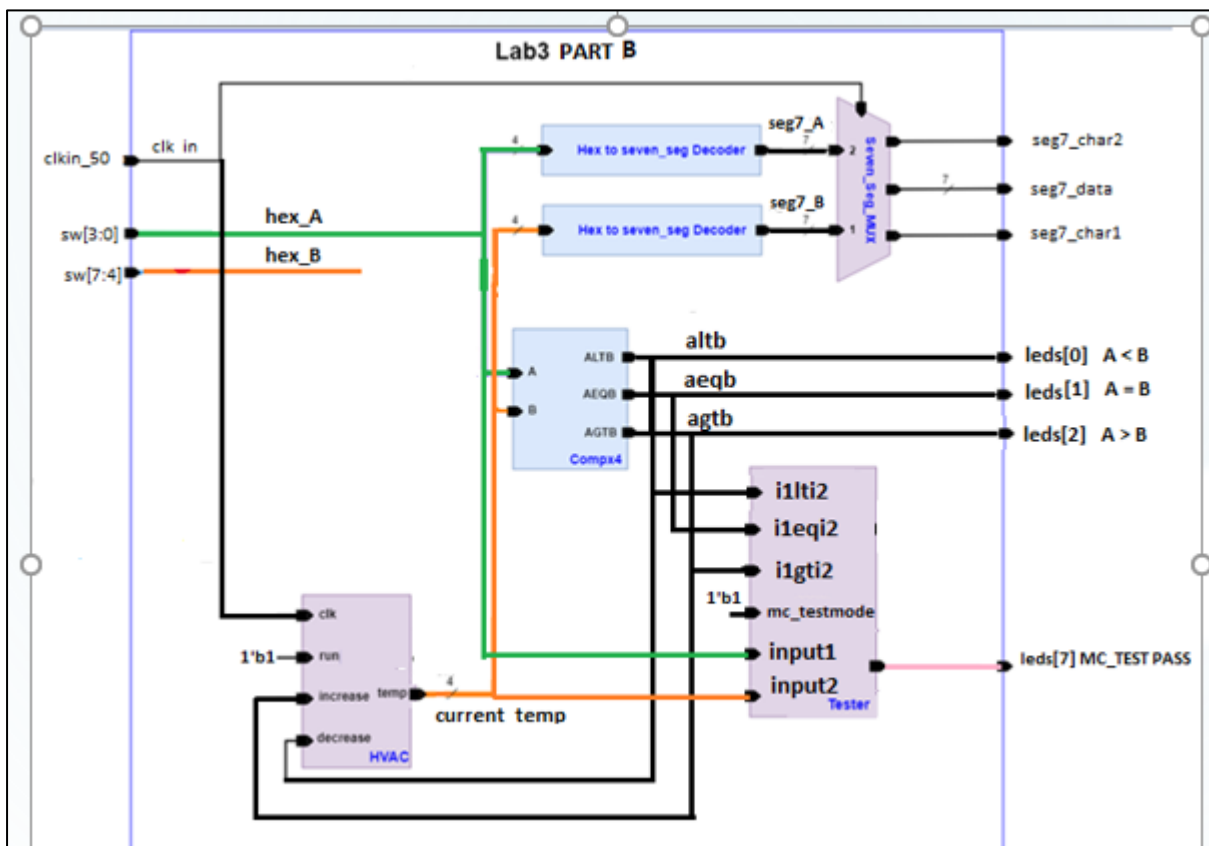
// COMMENT OUT THE comp4x U3 MODULE in the PART A Build section AND USE THIS ONE FOR PART B <-----
Comp4x U3 (
    .hex_A (hex_A), .hex_B (current_temp), .A_eq_B (aeqb), .A_gt_B (agtb), .A_lt_B (altb)
);

//add your 4bit up/down counter here
hvac #(1'b0) U4 (
    .clk (clk_in), .run (1'b1), .increase (agtb), .decrease (altb),
    .temp (current_temp)
);

//add your 4bit Tester here
Tester U5 (
    .mc_testmode (1'b1),
    .i1eqi2 (aeqb), .i1gti2 (agtb), .i1lti2 (altb),
    .input1 (hex_A), .input2 (current_temp),
    .test_pass (leds[7])
);
//-----
```

REMEMBER TO COMMENT OUT THE EARLIER PARTS MENTIONED ABOVE.
YOUR DESIGN SHOULD LOOK LIKE THE NEXT SLIDE WHEN DONE:

The graphic below shows what the design of the LogicalStep_Lab3_top should represent for PART B.



The hex_A temperature is the TARGET or Desired temperature and will appear on DIGIT2. The HVAC output is the CURRENT Temperature and will appear on DIGIT1. The PART B Build will have the HVAC CURRENT Temp always change to match the Desired Temp.

1.3.8 Lab3 Topic: Compiler Directives and Parameter Passing use in a Design

Compiler Directives in HDL (Hardware Description Language) designs can be of tremendous use for various situations. In particular, the **ifdef** and **ifndef** directives can be used to enable/disable the inclusion of specific logic blocks in a design. These directives use the “presence” of a macro name in a design file.

The Compiler directive begins with a ``` character (or grave accent). On the computer keyboards in the lab it is on the same key as the `~`.

To create a macro name, use the following command:

```
`define <macro name>
```

Then at appropriate points in your design file use the commands **`ifdef** (for when the macro name is present) or **`ifndef** (for when the macro name is NOT present). For either of those two commands, enter the Verilog statements and then terminate the list of statements with an **`endif** directive.

Multiple uses of these directives can be used for a design. These directives can be used in a Module Declaration section or in the logic design area.

An example is shown in the Part C Build where a directive macro named HVAC_SIM is used to add extra output ports (for simulation only) and also to specify connections to those ports and to pass a parameter value to the hvac module clock speed control.

1.3.9 Lab3 Part C: Employing Compiler Directives and adding a 2_to_1 MUX, 4-bit Inverter Modules

If you need to DEBUG your design, you may need to get access to internal signals within the design so that further design activity can be observed. The limitation that the FPGA on the LogicalStep board has is that ALL pins are presently used. Adding more pins to observe other internal signals will cause a Compiler error because no FPGA pins are left. But if we just add the extra pins for a synthesized design (in Quartus memory) there is never a compiler error because the extra pins don't have to be “fitted”, just “modeled”.

To add the pins for a simulation model only, the following directive can be inserted at the top of the file (above the Module declaration statement):

```
`define HVAC_SIM
```

Then in the middle of the module declaration statement, add the following:

```
`ifdef HVAC_SIM
    output [3:0]   hvac_temp,
`endif
```

Further on in the design, another set of compiler directives can be inserted to make connections to these ports and to set a parameter (hvac_sim) for passing to the hvac module.

```
`ifdef HVAC_SIM
    assign hvac_temp = current_temp;
    localparam hvac_sim = 1'b1;
`endif
`ifndef HVAC_SIM
    localparam hvac_sim = 1'b0;
`endif
```

When the hvac_temp pins are NOT to be included (ie: for a compile for a download to the FPGA), the `define HVAC_SIM can be merely “commented out” at the top of the file.

Below, is what is to be done in the PART C Build. **DON'T FORGET TO COMMENT-OUT SPECIFIC PORTIONS OF THE DESIGN AS SPECIFIED BELOW. THEN DO A FULL COMPILE for FPGA DOWNLOAD.**

```
//-----
// Part C build
// COMMENT OUT THE SEVENSEGMENT U0 MODULE in the starter part of the file AND USE THIS ONE FOR PART C  <-----
SevenSegment U0 (
    .hex (mux_temp), .sevenseg (seg7_A)
);

//COMMENT OUT the hvac module in the PART B section  <-----
hvac #(hvac_sim (hvac_sim)) U4 (
    .clk (clk_in), .run (1'b1), .increase (agtb), .decrease (altb),
    .temp (current_temp)
);
`ifdef HVAC_SIM
    assign hvac_temp = current_temp;
    localparam hvac_sim = 1'b1;
`endif
`ifndef HVAC_SIM
    localparam hvac_sim = 1'b0;
`endif
// COMMENT OUT THE comp4x U3 MODULE in the PART B Build section AND USE THIS ONE FOR PART D  <-----
comp4x U3 (
    .hex_A (mux_temp), .hex_B (current_temp),
    .A_eq_B (aeqb), .A_gt_B (agtb), .A_lt_B (altb)
);

//COMMENT OUT the Tester U5 module in the PART B section  <-----
Tester U5 (
    .mc_testmode (pb[2]),
    .ileq12(aeqb), .ilgt12(agtb), .ilgt12(altb),
    .input1 (hex_A), .input2 (current_temp),
    .test_pass (leds[7])
);

wire [3:0] mux_temp;

// add your input multiplexer here
mux_2to1_4bit U6 (
    .selector(pb[3]),
    .inp1(hex_A), .inp2 (hex_B),
    .muxout (mux_temp)
);

wire [3:0] pb;

// add your PB inverter block here
PB_Inverters U7 (
    .pbin (pb_n),
    .pbout(pb)
);

//-----

`define HVAC_SIM // COMMENT THIS LINE OUT IF DOING COMPILE FOR FPGA DOWNLOAD

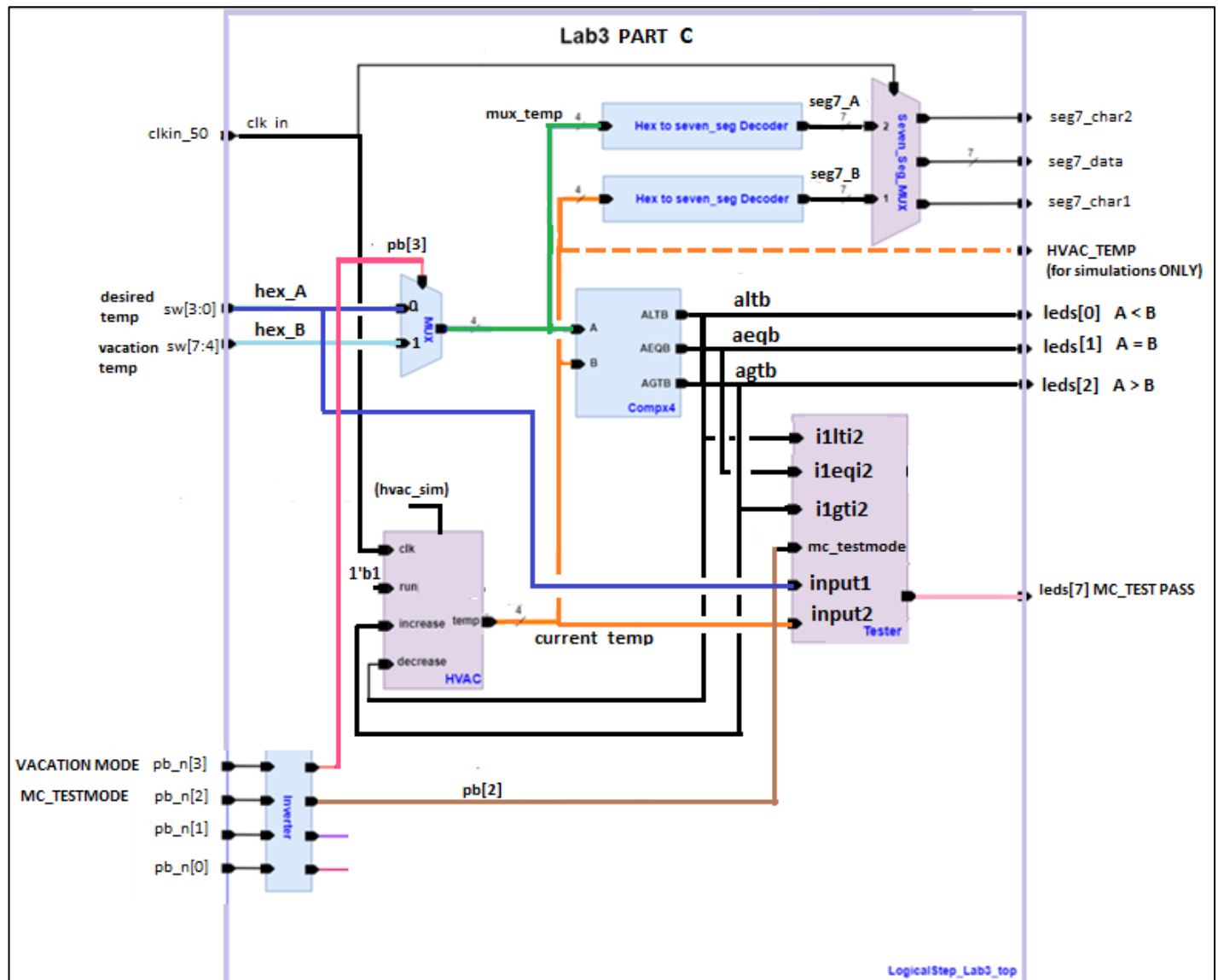
module LogicalStep_Lab3_top (
    input      clk_in_50,
    input      [3:0] pb_n,
    input      [7:0] sw,
    output     [7:0] leds,

    `ifdef HVAC_SIM
        output [3:0] hvac_temp,
    `endif

    output     [6:0] seg7_data, // 7-bit outputs to a 7-segment
    output     seg7_char1, // seg7 digit1 selector
    output     seg7_char2 // seg7 digit2 selector
);
```

The hex_A and hex_B signals connect to two the input ports of the 2_to_1 MUX. A selector for them comes from PB3.

A 4-bit inverter module will be added to invert the 4 pb_n signals. Your design should now be represented as the following:



For testing the PART C Build, **COMMENT-OUT** the `'define HVAC_SIM` compiler directive. Do a FULL Compile and then download it to the FPGA. The PART C Build can be tested by setting two different input temperatures. The hex_A can be set for the normal Desired Temp value and the hex_B can be set for the Vacation Temp value. Pressing PB3 will make the MUX-TEMP switch between these two values.

The PART C Build will have the HVAC CURRENT Temp always change to match the mux_temp.

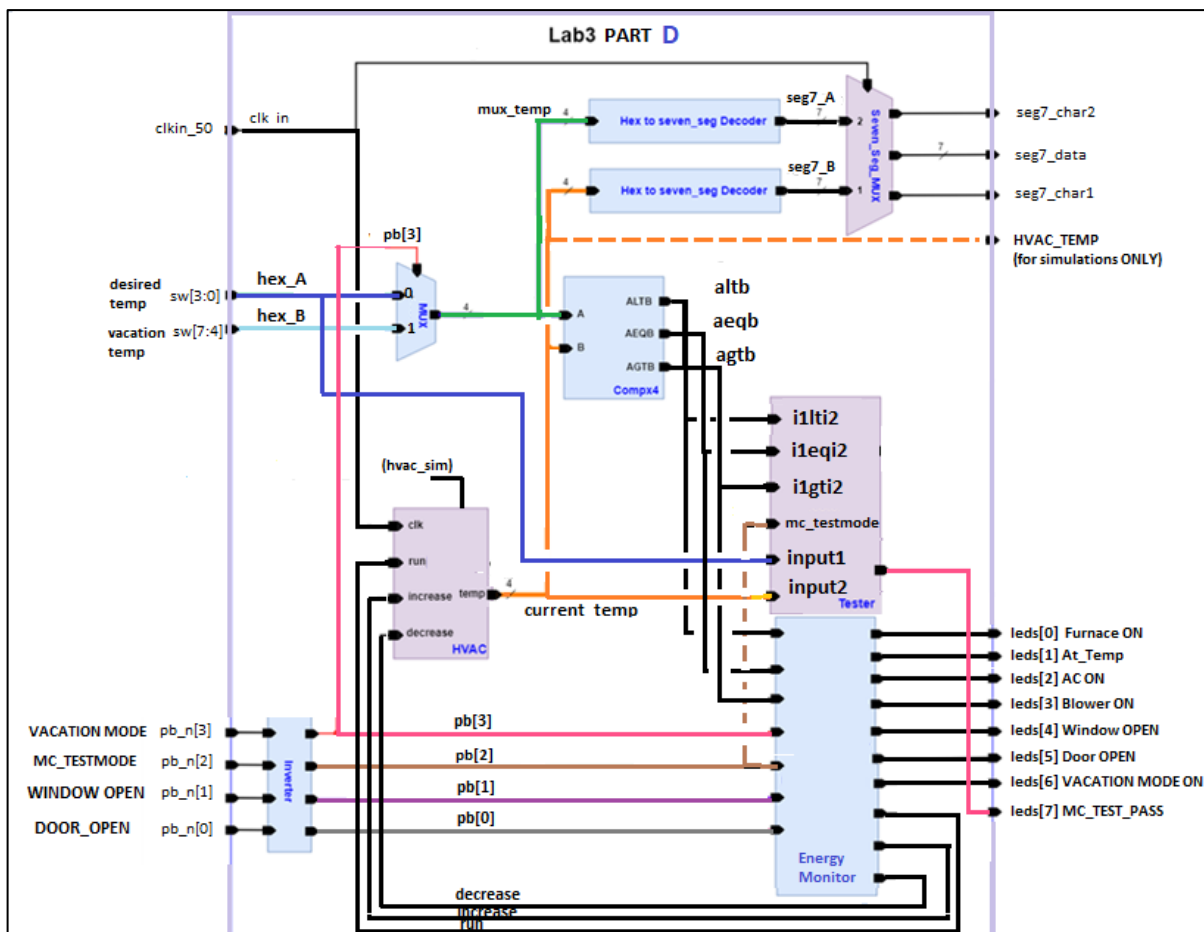
The mux_temp appears on DIGIT2 and the HVAC temperature appears on DIGIT1.

1.3.10 Lab3 Part D: Project – Creating a Home Energy Monitor (for the Lab3 Demo)

After adding the above code after the PART C Build section of the LogicalStep_Lab3_top.v file, the project design for this lab session is nearly complete. It is a Verilog-based FPGA design for implementing a simple Energy Monitoring System that could be used in a home.

```
//-----
// Part D build
// COMMENT OUT ASSIGN STATEMENTS IN PART A FOR leds[2], leds[1], leds[0]
//COMMENT OUT the hvac module in the PART C section
hvac #(.hvac_sim (hvac_sim)) U4 (
  .clk (clk_in), .run (run), .increase (increase), .decrease (decrease),
  .temp (current_temp)
);
//add declarations associated with the Energy Monitor Control block signals
wire increase, decrease, run;
//add Energy Monitor Control block here
Energy_Monitor_Control U8 (
  .door_open(pb[0]), .window_open(pb[1]), .mc_testmode(pb[2]), .vac_mode(pb[3]),
  .i1eqi2(aeqb), .i1gti2(agtb), .i1lti2(altb),
  .blower_on(leds[3]), .ac_on(leds[2]), .at_temp(leds[1]), .furnace_on(leds[0]),
  .HVAC_run(run), .HVAC_increase(increase), .HVAC_decrease(decrease),
  .vacation_led(leds[6]), .door_open_led(leds[5]), .window_open_led(leds[4])
);
//-----
endmodule
```

The updated top-level graphic is shown below. The connections at the top-level are now done.



An Energy Monitor Control block is the new module added to the design in Part D. The Starting version of the Energy_Monitor_Control.v file is below:

```

module Energy_Monitor_Control (
    input door_open, window_open, mc_testmode, vac_mode,
    input ileqi2, ilgti2, illti2,

    output blower_on, ac_on, furnace_on, at_temp,
    output HVAC_run, HVAC_increase, HVAC_decrease,
    output Vacation_led, door_open_led, window_open_led
);

endmodule

```

It is the central function that observes various inputs and activates outputs for the functioning of the system. You must develop the Energy_Monitor_Control.v design. You may use any style you wish (Dataflow/Structural/Behavioural) in that file. Below are the assignments of signals to the FPGA ports for the Lab3 project:

TEMPERATURE Control INPUTS (Range from 0 to F)	desired_temp[3..0]	sw[3..0]
	vacation_temp[3..0]	sw[7..4]
OPERATION MODE INPUTS	vacation_mode	pb_n[3]
	MC_test_mode	pb_n[2]
SENSOR INPUTS	window_open	pb_n[1]
	door_open	pb_n[0]
OUTPUT INDICATORS	furnace_on	leds[0]
	at_temp	leds[1]
	AC_on	leds[2]
	blower_ON	leds[3]
	window_open_led	leds[4]
	door_open_led	leds[5]
	Vacation_led	leds[6]
	MC_test_pass	leds[7]
	Mux_temp_7seg[3..0]	DIGIT2
	Current_temp_7seg[3..0]	DIGIT1
OUTPUT For sim's only (comment out for FPGA download compiles)	current__temp[3:0] (internal)	Virtual pins HVAC_temp[3:0]

Some Lab3 Energy Monitoring System Operational Requirements:

- 1) The Energy Monitor Control block shall direct the HVAC Unit to increase or decrease the temperature depending on the outputs coming from the Comp4 comparator. This shall include driving the “run” signal to the active state. When the “current_temp” equals the “mux_temp” the Energy Monitor Control shall deactivate the “run” signal. With “run” being OFF the “decrease” or “increase” inputs to the HVAC are ignored.
- 2) The “run” signal to the HVAC shall NOT be active if either the “window_open” or “door_open” sensors are detected as being ON
- 3) The “run” signal to the HVAC shall NOT be active if the “MC_test_mode” (PB[2] for testing the comparator) is ON.
- 4) The indicator for “~~furnace_ON~~ AC_on” shall be active when the “mux_temp” is lesser than “current_temp”.
- 5) The indicator for “~~AC_on~~ furnace_ON” shall be active when the “mux_temp” is greater than “current_temp”.
- 6) The indicator for “at_temp” shall be active when the “mux_temp” = “current_temp” and the “blower_ON” led shall be **OFF**.
- 7) The indicator for “blower_ON” shall be ON when “mux_temp” is different than “current_temp” **except for when any of the MC_test_mode (PB[2] is ON), Window_OPEN (PB[1] is ON) or Door_OPEN (PB[0] is ON).**
- 8) The indicator for “vacation” shall be active when the “vacation_mode” pb is detected as ON.

The “desired_temp” setting and “vacation_temp” are multiplexed. The selector for the mux is the “vacation_mode (PB[3] is ON) input. When active, the “vacation_temp” is presented to the “mux_temp” signal. In Normal Mode the output of the MUX presents the “desired_temp” value.

The Comparator will be tested by the Tester block when the “MC_test_mode” input is active. It will compare the “**desired_temp**” value against the “current_temp” value. With this setup, if the Comparator is working correctly, the Tester will activate the “MC_test_pass” indicator. Otherwise, for a test failure detection, the Tester will keep the “MC_test_pass” indicator OFF.

To force a mismatch between the test bench comparisons with the Magnitude Comparator there is a way to do this using the “vacation_mode”, “vacation_temp”, “desired_temp” and the “MC_test_mode” together.

In general, all functionality of designs must be tested thoroughly.

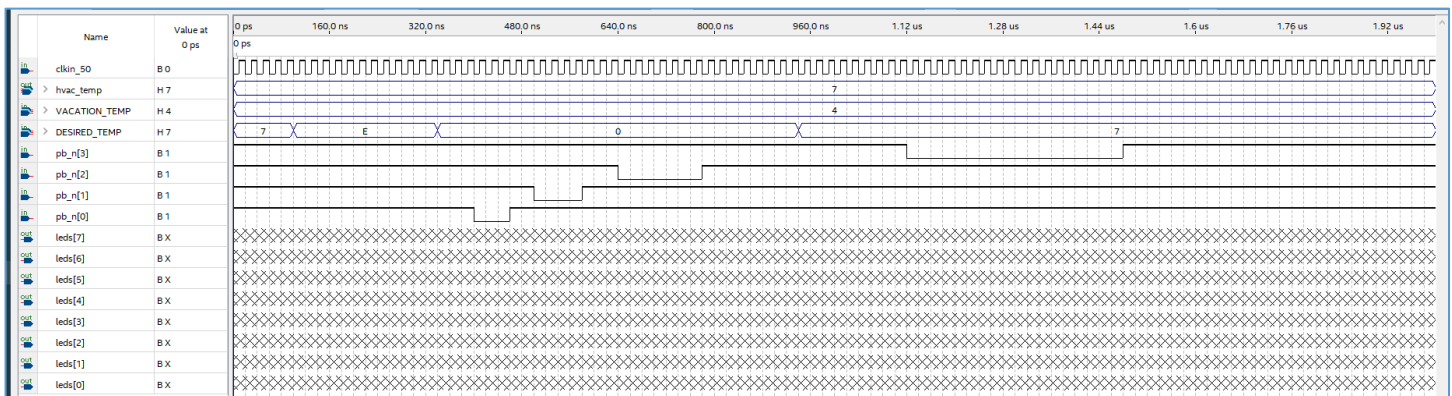
How can the Magnitude Comparator Tester be made to detect a fail?

This will be something that you will be asked to do during your Lab3 Demo in the Lab.

For simulations of your design, you may uncomment the compiler directive at the top of the file
``define HVAC_SIM`

For simulations you can run an Analysis & Synthesis compile to create a simulation model in Quartus memory. Remember that the HVAC_temp[3:0] is available in the synthesized design for simulation. The clock speed going to the HVAC counter is 50 MHz (20 period) for simulations.

Test your inputs in the simulation stimulus to confirm that the HVAC is properly operated and controlled. An example of a simulation stimulus might be like the following:



If you need to add more internal signals you can add more ports in the ``ifdef HVAC_SIM` part of the Module Declaration of the top-level file. Then add more signals to the ``ifdef HVAC_SIM` part further on in the file to connect to the new ports.

To return to FPGA downloading just “comment-out” the ``define HVAC_SIM` line at the top of the file.

Be prepared to show and BRIEFLY explain your simulation during the Lab3 Demo

